

Edge AI Fall Detection System: A Hardware vs. Software Architecture Comparison

Embedded Systems Mini-Project

1st Yossi Brim

*Electrical and Electronics Engineering
Jerusalem College of Technology (JCT)
Jerusalem, Israel*

2nd Elad Asbag

*Electrical and Electronics Engineering
Jerusalem College of Technology (JCT)
Jerusalem, Israel*

Abstract—Fall detection systems are critical for the safety of elderly individuals and dementia patients. While hardware-accelerated solutions utilizing Field-Programmable Gate Arrays (FPGAs) offer deterministic, real-time performance, they often entail high costs, complex memory management, and prolonged development cycles. This paper presents a software-based Edge Computing alternative using a Raspberry Pi and a pre-trained computer vision model (MediaPipe). We propose a lightweight fall detection pipeline that processes video streams, extracts kinematic skeletal landmarks, and communicates alert states via an I2C-based hardware display. The objective of this mini-project is to provide a Proof of Concept (PoC) and conduct a comparative analysis between the software-based Edge AI architecture and a traditional FPGA/RTL implementation in terms of development agility, hardware complexity, and overall efficiency.

I. INTRODUCTION

The engineering challenge of detecting human falls in real-time is typically addressed using dedicated hardware systems to ensure low latency and high reliability. In our parallel senior design project, a fall detection system is implemented on an Artix-7 FPGA, leveraging custom RTL video pipelines, BRAM memory buffers, and bare-metal SCCB/VGA controllers. While this architecture provides uncompromising deterministic performance, it significantly increases the Time-to-Market (TTM) and engineering overhead.

In modern embedded systems, there is a growing paradigm shift towards Edge AI—running optimized machine learning models on accessible microcomputers. This project investigates the feasibility of replacing the complex hardware pipeline with a software-centric approach. By utilizing a Raspberry Pi and a pre-trained Pose Estimation deep learning model, we aim to extract spatial body coordinates and apply deductive fall-detection logic in Python. Furthermore, to satisfy physical hardware interfacing requirements, the system acts as an I2C master bus, transmitting real-time alerts to a digital display. This paper explores the system architecture, the underlying AI algorithm, and evaluates the trade-offs of Edge Computing versus FPGA acceleration.

A. Architectural Comparison: Hardware (FPGA) vs. Software (Edge AI)

In developing the Edge AI software equivalent to the RTL-based Artix-7 implementation, two primary engineering challenges were addressed: resource management and temporal noise filtering.

1) *Spatial Processing and Resource Optimization*: In the FPGA environment, real-time video processing is heavily constrained by available Block RAM (BRAM) and DSP slices. Consequently, hardware implementations often require manual binarization thresholds and spatial downsampling to reduce high-resolution RGB data. Conversely, the software PoC abstracts this complexity using Google's pre-trained MediaPipe framework. By initializing the AI core with `model_complexity=0`, the framework automatically scales and optimizes the input matrices, allowing the Raspberry Pi 3 CPU to maintain a viable frame rate without explicit manual pixel decimation.

2) *Temporal Noise Filtering (FSM Debouncing)*: Both implementations necessitate a mechanism to filter transient visual anomalies (e.g., a single distorted frame causing a false positive). In the VHDL implementation, this is managed via a synchronous counter within the Finite State Machine (FSM), asserting the detection flag only after the condition persists for a predefined threshold of clock cycles. The software PoC mirrors this exact logic utilizing a frame-based hysteresis threshold. The top-level controller (`main.py`) tracks consecutive frames yielding a positive geometric pose estimation. The hardware I2C display alert is triggered exclusively when `consecutive_falls ≥ 3`, ensuring high reliability and eliminating brief temporal glitches.

II. METHODOLOGY AND AI ALGORITHM

III. RESULTS AND PERFORMANCE

IV. CONCLUSION